```
import os
import tensorflow as tf

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score

from sys import getsizeof
import cv2
```

Outputting size

```
def get_file_size(file_path):
    size = os.path.getsize(file_path)
    return size

def convert_bytes(size, unit=None):
    if unit == "KB":
        return print('File size: ' + str(round(size / 1024, 3)) + '
Kilobytes')
    elif unit == "MB":
        return print('File size: ' + str(round(size / (1024 * 1024),
3)) + ' Megabytes')
    else:
        return print('File size: ' + str(size) + ' bytes')
```

Loading datasets

```
main_dir = '/content/drive/MyDrive/Colab Notebooks/week8/'
data_dir = '/content/drive/MyDrive/Colab Notebooks/week8/val'

labels = sorted(os.listdir(data_dir))
print(labels)
```

```
['Black-grass', 'Charlock', 'Cleavers', 'Common Chickweed', 'Common
wheat', 'Fat Hen', 'Loose Silky-bent', 'Maize', 'Scentless Mayweed',
'Shepherds Purse', 'Small-flowered Cranesbill', 'Sugar beet']
```

```
eval_data = []
for ids, label in enumerate(labels):
    for file in os.listdir(os.path.join(data_dir, label)):

        eval_data.append(['val/{}/{}'.format(label, file), ids,
label])


eval = pd.DataFrame(eval_data, columns=['file', 'id','labels'])
eval.head()
```

```
                            file  id       labels
0  val/Black-grass/0ace21089.png   0  Black-grass
1  val/Black-grass/0b3e7a7a9.png   0  Black-grass
```

```
2  val/Black-grass/0bb75ded8.png   0  Black-grass
3  val/Black-grass/0b228a6b8.png   0  Black-grass
4  val/Black-grass/0be707615.png   0  Black-grass
```

```
IMAGE_SIZE = 120

def read_image(filepath):
    return cv2.imread(os.path.join(main_dir,filepath))

def resize_image(img,img_size):
    return
cv2.resize(img.copy(),img_size,interpolation=cv2.INTER_AREA)

X_eval = np.zeros((eval.shape[0], IMAGE_SIZE, IMAGE_SIZE, 3))
for i, file in enumerate(eval['file'].values):
    #print(file)
    image = read_image(file)
   # print(image)
    X_eval[i] = resize_image(image, (IMAGE_SIZE, IMAGE_SIZE))
# Normalize the data
x_eval = X_eval / 255.
print('eval Shape: {}'.format(X_eval.shape))
```

```
eval Shape: (120, 120, 120, 3)
```

```
y_eval = eval['labels']
y_eval.shape
```

```
(120,)
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_eval = le.fit_transform(y_eval)

y_eval[:5]
```

```
array([0, 0, 0, 0, 0])
```

```
y_eval = tf.keras.utils.to_categorical(y_eval)

model = tf.keras.models.load_model('/content/drive/MyDrive/Colab
Notebooks/week8/inc.h5')

X_eval = X_eval/255.0

model.evaluate(X_eval,y_eval)
```

```
4/4 [==============================] - 11s 427ms/step - loss: 3.4757 -
accuracy: 0.2833

[3.475726842880249, 0.28333333134651184]
```

```
convert_bytes(get_file_size('/content/drive/MyDrive/Colab
Notebooks/week8/inc.h5'), "MB")
```

```
{"type":"string"}
```

TFlite

```
TF_LITE_MODEL_FILE_NAME = "tf_lite_model.tflite"
```

```
tf_lite_converter = tf.lite.TFLiteConverter.from_keras_model(model)
tf_lite_converter.optimizations = [tf.lite.Optimize.OPTIMIZE_FOR_SIZE]
# tf_lite_converter.optimizations = [tf.lite.Optimize.DEFAULT]

tflite_model = tf_lite_converter.convert()
```

```
WARNING:absl:Optimization option OPTIMIZE_FOR_SIZE is deprecated,
please use optimizations=[Optimize.DEFAULT] instead.

INFO:tensorflow:Assets written to: /tmp/tmpjxnhcg86/assets

INFO:tensorflow:Assets written to: /tmp/tmpjxnhcg86/assets
WARNING:absl:Optimization option OPTIMIZE_FOR_SIZE is deprecated,
please use optimizations=[Optimize.DEFAULT] instead.
WARNING:absl:Optimization option OPTIMIZE_FOR_SIZE is deprecated,
please use optimizations=[Optimize.DEFAULT] instead.
WARNING:absl:Buffer deduplication procedure will be skipped when
flatbuffer library is not properly loaded
```

```
tflite_model_name = TF_LITE_MODEL_FILE_NAME
open(tflite_model_name, "wb").write(tflite_model)
```

```
19810576
```

```
convert_bytes(get_file_size(TF_LITE_MODEL_FILE_NAME), "MB")
```

```
File size: 18.893 Megabytes
```

Check input tensor

```
interpreter = tf.lite.Interpreter(model_path =
TF_LITE_MODEL_FILE_NAME)
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])
```

```
Input Shape: [  1 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [ 1 12]
Output Type: <class 'numpy.float32'>
```

```
interpreter.resize_tensor_input(input_details[0]['index'], (120, 120,
120,3))
interpreter.resize_tensor_input(output_details[0]['index'], (120, 120,
120,3))
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])
```

```
Input Shape: [120 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [120  12]
Output Type: <class 'numpy.float32'>
```

```
X_eval.dtype
```

```
dtype('float64')
```

```
X_eval_numpy = np.array(X_eval, dtype=np.float32)
X_eval_numpy.dtype
```

```
dtype('float32')
```

```
interpreter.set_tensor(input_details[0]['index'], X_eval_numpy)
interpreter.invoke()
tflite_model_predictions = interpreter.get_tensor(output_details[0]
['index'])
print("Prediction results shape:", tflite_model_predictions.shape)
prediction_classes = np.argmax(tflite_model_predictions, axis=1)
```

```
Prediction results shape: (120, 12)
```

```
#acc = accuracy_score(prediction_classes, y_eval)
prediction_classes_onehot =
tf.keras.utils.to_categorical(prediction_classes)
prediction_classes_onehot
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 1., 0.],
       [0., 0., 0., ..., 0., 0., 0.]], dtype=float32)
```

```
y_eval
```

```
array([[1., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
```

```
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.]], dtype=float32)

acc = accuracy_score(prediction_classes_onehot, y_eval)

print('Test accuracy TFLITE model :', acc)

Test accuracy TFLITE model : 0.24166666666666667
```

For fp16

```
fp16_tf_lite_converter =
tf.lite.TFLiteConverter.from_keras_model(model)
fp16_tf_lite_converter.target_spec.supported_types = [tf.float16]
fp16_tflite_model = fp16_tf_lite_converter.convert()

INFO:tensorflow:Assets written to: /tmp/tmpvltr1p4r/assets

INFO:tensorflow:Assets written to: /tmp/tmpvltr1p4r/assets
WARNING:absl:Buffer deduplication procedure will be skipped when
flatbuffer library is not properly loaded

fp16_TF_LITE_MODEL_FILE_NAME = "fp16_tf_lite_model.tflite"

fp16_tflite_model_name = fp16_TF_LITE_MODEL_FILE_NAME
open(fp16_tflite_model_name, "wb").write(fp16_tflite_model)

78807764

convert_bytes(get_file_size(fp16_TF_LITE_MODEL_FILE_NAME), "MB")

File size: 75.157 Megabytes

interpreter = tf.lite.Interpreter(model_path =
fp16_TF_LITE_MODEL_FILE_NAME)
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])

Input Shape: [  1 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [ 1 12]
Output Type: <class 'numpy.float32'>

interpreter.resize_tensor_input(input_details[0]['index'], (120, 120,
120,3))
interpreter.resize_tensor_input(output_details[0]['index'], (120, 120,
120,3))
interpreter.allocate_tensors()
```

```
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])
```

```
Input Shape: [120 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [120  12]
Output Type: <class 'numpy.float32'>
```

```
interpreter.set_tensor(input_details[0]['index'], X_eval_numpy)
interpreter.invoke()
tflite_model_predictions = interpreter.get_tensor(output_details[0]
['index'])
print("Prediction results shape:", tflite_model_predictions.shape)
prediction_classes = np.argmax(tflite_model_predictions, axis=1)
```

```
Prediction results shape: (120, 12)
```

```
prediction_classes_onehot =
tf.keras.utils.to_categorical(prediction_classes)
```

```
acc = accuracy_score(prediction_classes_onehot, y_eval)
acc
```

```
0.2833333333333333
```

For fp 32

```
fp32_TF_LITE_MODEL_FILE_NAME = "fp32_tf_lite_model.tflite"
```

```
fp32_tf_lite_converter =
tf.lite.TFLiteConverter.from_keras_model(model)
fp32_tf_lite_converter.target_spec.supported_types = [tf.float32]
fp32_tflite_model = fp32_tf_lite_converter.convert()
```

```
INFO:tensorflow:Assets written to: /tmp/tmpp5iyugf9/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmpp5iyugf9/assets
WARNING:absl:Buffer deduplication procedure will be skipped when
flatbuffer library is not properly loaded
```

```
fp32_tflite_model_name = fp32_TF_LITE_MODEL_FILE_NAME
open(fp32_tflite_model_name, "wb").write(fp32_tflite_model)
```

```
78807764
```

```
convert_bytes(get_file_size(fp32_TF_LITE_MODEL_FILE_NAME), "MB")
```

```
File size: 75.157 Megabytes
```

```
interpreter = tf.lite.Interpreter(model_path =
fp32_TF_LITE_MODEL_FILE_NAME)
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])
```

```
Input Shape: [  1 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [ 1 12]
Output Type: <class 'numpy.float32'>
```

```
interpreter.resize_tensor_input(input_details[0]['index'], (120, 120,
120,3))
interpreter.resize_tensor_input(output_details[0]['index'], (120, 120,
120,3))
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Shape:", input_details[0]['shape'])
print("Input Type:", input_details[0]['dtype'])
print("Output Shape:", output_details[0]['shape'])
print("Output Type:", output_details[0]['dtype'])
```

```
Input Shape: [120 120 120   3]
Input Type: <class 'numpy.float32'>
Output Shape: [120  12]
Output Type: <class 'numpy.float32'>
```

```
interpreter.set_tensor(input_details[0]['index'], X_eval_numpy)
interpreter.invoke()
tflite_model_predictions = interpreter.get_tensor(output_details[0]
['index'])
print("Prediction results shape:", tflite_model_predictions.shape)
```

```
Prediction results shape: (120, 12)
```

```
fp32_prediction_classes = np.argmax(tflite_model_predictions, axis=1)
```

```
fp32_prediction_classes_onehot =
tf.keras.utils.to_categorical(fp32_prediction_classes)
```

```
fp32_acc = accuracy_score(fp32_prediction_classes_onehot, y_eval)
acc
```

```
0.2833333333333333
```